

Definición y esquemas para el Simulador

CLUB DE VUELO UPM

Pablo Martín Yuste

20 de marzo de 2026

1. Aspectos generales del simulador

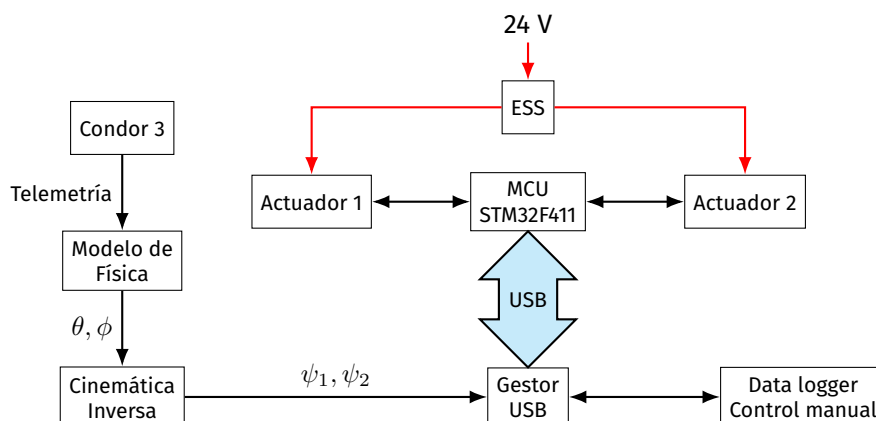
El simulador consiste en el fuselaje de la cabina de un planeador **ToDo 1: Foto de un modelo del planeador del simu**, montada sobre una junta universal que permite los giros de cabeceo y alabeo, y dotado de dos motores eléctricos que mueven las articulaciones a la parte posterior del fuselaje.

El software empleado es **Condor 3**, que permite una extracción de los datos del estado de vuelo del planeador mediante el protocolo UDP.

Este proyecto implementa un software para la extracción, interpretación y procesado de la telemetría de Condor 3, y lo combina con un hardware para controlar dinámicamente la actitud del planeador, con el objetivo de ofrecer una experiencia de vuelo realista.

2. Sistemas del simulador

El funcionamiento general del simulador se explica a continuación:



Como se puede observar, los datos brutos extraídos del Condor 3 pasan por dos tipos de procesado. En primer lugar la telemetría se convierte en unos ángulos de cabeceo y alabeo para el simulador real, utilizando un modelo de física que busca reproducir de la forma más fiel posible las fuerzas que se sentirían en cabina en las condiciones del vuelo simulado.

A continuación, un cálculo de cinemática inversa resuelve a qué posiciones se deben mover cada uno de los motores para alcanzar esa posición. Una vez determinadas estas posiciones, estos datos son pasados al gestor USB mediante un ciclo de control de alta frecuencia, para mover los motores.

El microcontrolador es responsable de implementar un algoritmo de control **ToDo 2: referencia cuando se explique este algoritmo** para mover los motores. Cada uno de los motores está instrumentado con un sensor de ángulo magnético para conocer exactamente su posición, permitiendo un control en bucle cerrado.

Como medida de protección activa, está presente un botón de desconexión de emergencia (ESS, *Emergency Stop Switch*), que desactivará los puentes H e interrumpirá físicamente el suministro de corriente a los motores, permitiendo parar instantáneamente el movimiento del simulador. Además, el microcontrolador está conectado a este botón, indicando al software de escritorio esta detención.

2.1. Aplicación de escritorio

Para implementar de forma centralizada todo el código, se ha desarrollado una aplicación de escritorio responsable de preprocesar los datos de telemetría de Condor 3, y convertirlos en comandos de posición que enviar al sistema de control de los actuadores. Esta aplicación se ha desarrollado en Python, por sus facilidades para el desarrollo de la interfaz gráfica, con módulos individuales programados en C++. A continuación se explica detalladamente cada uno de sus componentes.

2.1.1. Interfaz Gráfica

Para facilitar el uso del software, se ha desarrollado una interfaz gráfica de uso general, que proporciona controles útiles, como activar o desactivar el sistema de movimiento, indicar si la telemetría se recibe correctamente de Condor 3, o controlar manualmente el simulador.

2.1.2. Modelo de física

ToDo 3: Escribir detalladamente el funcionamiento del modelo de física

Programado en C++ para reducir el tiempo de ejecución y aumentar la fiabilidad del código.

Se debe explicar qué datos se extraen de la telemetría, por qué son útiles, en qué ejes están expresados, cómo se manipulan, qué tipos de algoritmos de control se han empleado, si hay washout o no, qué limitaciones hay en las interfaces (ángulos máximos, velocidades máximas) etc.

Idealmente en el departamento de Mecánica del Vuelo me pueden explicar algo de cómo hacer esto...

2.1.3. Cinemática inversa

ToDo 4: Explicar la resolución de la cinemática inversa

Programado en C++ para reducir el tiempo de ejecución y aumentar la fiabilidad del código.

Puede ser interesante explorar cómo se podrían encontrar los límites del movimiento del simulador, establecer los distintos criterios para encontrarlos, y programar en esta parte del código fail-safes.

2.1.4. Gestor USB

Esta parte de la aplicación de escritorio desarrollada se encarga de procesar los datos que deben transmitirse por la comunicación USB y recibir la información de estado que el microcontrolador devuelve. Se ha programado de forma independiente para poder gestionar eficientemente altos volúmenes de información.

ToDo 5: Decidir si programar en Python o en C++.

2.1.5. Comunicación USB

ToDo 6: Definir comunicación, qué datos van desde la app de escritorio, qué datos manda el microcontrolador de vuelta, cuál es la latencia objetivo de la comunicación, etc.

2.2. Sistema de control del movimiento

Externo a la aplicación de escritorio, y utilizando el protocolo USB **ToDo 7: Esta terminología es correcta?** como comunicación, el sistema de control del movimiento consiste en toda la electrónica encargada de garantizar el movimiento de los actuadores de forma precisa y rápida. Los componentes se describen a continuación.

Se ha considerado como actuadores a todo el conjunto de puente H, motor y encoder magnético que está duplicado en el simulador. Cuando está conectado todo, el puente H recibe una señal PWM del microcontrolador que determina el par (y la velocidad) con la que el motor busca alcanzar la posición comandada.

2.2.1. Microcontrolador

ToDo 8: Explicar qué código se ejecuta en el microcontrolador, y qué algoritmos se emplean para el control de los motores (actuadores), si se implementa (o no) la regulación de la intensidad máxima...

Se ha elegido el STM32F411 por su alta frecuencia del microprocesador, además de tener disponibles dos buses I2C independientes, permitiendo una comunicación de milisegundos con los encoders magnéticos en paralelo.

ToDo 9: añadir foto del microcontrolador

ToDo 10: Link a la hoja técnica del blackpill

2.2.2. Motores

Los motores son actuadores del parabrisas de un autobús, extraídos de un desguace. Se trata de motores de continua de 24 V. Son capaces de proporcionar 16 Nm de par, con una reducción mediante tornillo sinfín.

ToDo 11: Añadir foto de los motores

2.2.3. Encoders Magnéticos

Para determinar con precisión la posición de cada uno de los motores, se están utilizando encoders magnéticos MT6701. Estos se comunican mediante el protocolo I2C al microcontrolador, con una precisión de 14 bits en la comunicación.

ToDo 12: Foto de MT6701 montado y sin montar

[Datasheet](#) del sensor.

3. Consideraciones

3.1. Control de Intensidad

Experiencias pasadas con este sistema, utilizando algoritmos de control menos sofisticados, han demostrado que es frecuente que se exceda la intensidad máxima de trabajo de los actuadores, durante cambios bruscos de sentido (inversiones de carga). Esto puede provocar el colapso de los puentes H o daños en los devanados de los motores, por lo que es recomendable implementar instrumentación de medida de corriente, para añadir regulación en un futuro.

3.2. Resensorización del simulador

En el estado actual del simulador, las palancas físicas responsables del movimiento de los ejes de control está siendo gestionado por dos microcontroladores independientes: un LEO-BODNAR (antiguo) y un Arduino Pro Micro (más moderno). El LEO-BODNAR se encarga de los interruptores del tren de aterrizaje, mientras que el Arduino se encarga de los ejes de freno, guiñada y compensador.

Para evitar cableado y sistemas extra que no son necesarios, una posible mejora sería eliminar el controlador LEO-BODNAR y utilizar el Arduino Pro Micro para controlar todos los botones y ejes de la cabina. Además, los ejes cuya medida se hace ahora mediante potenciómetros, se deberían cambiar a sensores magnéticos de efecto de campo o encoders magnéticos, para aumentar la precisión y la vida de estos sensores.

La mejor propuesta para hacer esto consistiría en utilizar el multiplexor TCA9548A, conectado al Arduino para tener hasta 8 canales I2C con los que leer datos de encoders magnéticos, y considerar utilizar alguna de las matrices para botones que hay disponibles en el club.

Lista de cosas pendientes

1. Foto de un modelo del planeador del simu p. 1
2. referencia cuando se explique este algoritmo p. 2
3. Escribir detalladamente el funcionamiento del modelo de física p. 2
4. Explicar la resolución de la cinemática inversa p. 2
5. Decidir si programar en Python o en C++ p. 3
6. Definir comunicación p. 3
7. Esta terminología es correcta? p. 3
8. Explicar qué código se ejecuta en el microcontrolador p. 3
9. añadir foto del microcontrolador p. 3
10. Link a la hoja técnica del blackpill p. 3
11. Añadir foto de los motores p. 3
12. Foto de MT6701 montado y sin montar p. 3